

3D Data Processing - Lab 2

Experimentation Submission

How to use

Same code as the minimal submission (tasks 3 to 7 implemented in *basic_sfm.cpp*; tasks 1, 2, OPTIONAL 1, OPTIONAL 2 left as the lab template). The difference: four runtime knobs are exposed via environment variables so you can sweep configurations and pick the one whose .ply looks best to you.

1. Build (one time)

From the project root, the same procedure as any CMake project:

```
mkdir -p build
cd build
cmake -DCMAKE_BUILD_TYPE=Release ..
make -j$(nproc)
cd ..
```

After this, `./bin/basic_sfm` exists and is ready to use.

2. Default command

The plain command runs with the default configuration (Cauchy loss, scale = 2.0, sideward threshold = 0.5, divergence floor = 2.0). These defaults are exactly the values the minimal submission ships with and are directly compliant with the lab spec hint (e.g., `1.0` or `2 * max_reproj_err_`).

```
./bin/basic_sfm datasets/test_data1.txt cloud1.ply
./bin/basic_sfm datasets/test_data2.txt cloud2.ply
```

After each run open the produced `.ply` in MeshLab. To get a clean view, hide the trackball sphere with *View -> Show Trackball* and increase the point size in the right panel.

3. Available knobs (environment variables)

All three knobs are read at runtime by `basic_sfm`. Set them by prefixing the command. Multiple knobs can be combined.

Variable	Default	Range	Effect
SFM_LOSS	cauchy	{cauchy, huber, null}	Robust loss kernel applied to every reprojection residual in BA (task 6/7).
SFM_SCALE	2.0	any positive double	Scale factor for the loss kernel: actual scale is <code>SFM_SCALE * max_reproj_err_</code> .
SFM_SIDEWARD	0.5	0.0 to 1.0	Threshold in task 3/7. Pair is rejected if BOTH <code> tx </code> and <code> ty </code> are below this.

4. Example commands you can copy-paste

4.1 Default

```
./bin/basic_sfm datasets/test_data1.txt cloud1.ply
./bin/basic_sfm datasets/test_data2.txt cloud2.ply
```

4.2 Try Huber loss instead of Cauchy

Huber damps outliers more gently than Cauchy. In our tests it produced visibly denser DS1 reconstructions but a sparser DS2.

```
SFM_LOSS=huber ./bin/basic_sfm datasets/test_data1.txt cloud1_huber.ply
SFM_LOSS=huber ./bin/basic_sfm datasets/test_data2.txt cloud2_huber.ply
```

4.3 Try plain least squares (NULL loss)

No outlier downweighting. Often unstable when the matcher's RANSAC let some bad matches through; useful as the baseline that the spec asks you to compare against.

```
SFM_LOSS=null ./bin/basic_sfm datasets/test_data1.txt cloud1_null.ply
SFM_LOSS=null ./bin/basic_sfm datasets/test_data2.txt cloud2_null.ply
```

4.4 Vary the loss scale

Smaller scale = more aggressive outlier suppression; larger scale = behaviour closer to plain least squares.

```
SFM_LOSS=cauchy SFM_SCALE=1.0 ./bin/basic_sfm datasets/test_data1.txt cloud1_c1.ply
SFM_LOSS=cauchy SFM_SCALE=4.0 ./bin/basic_sfm datasets/test_data1.txt cloud1_c4.ply
SFM_LOSS=huber SFM_SCALE=1.0 ./bin/basic_sfm datasets/test_data1.txt cloud1_h1.ply
```

4.5 Loosen the sideward seed check

If basic_sfm rejects too many seed pairs as forward-dominant, lower the threshold. 0.3 is permissive, 0.7 is strict.

```
SFM_SIDEWARD=0.3 ./bin/basic_sfm datasets/test_data1.txt cloud1_sw03.ply
SFM_SIDEWARD=0.7 ./bin/basic_sfm datasets/test_data1.txt cloud1_sw07.ply
```

4.6 Combine multiple knobs

```
SFM_LOSS=huber SFM_SCALE=1.5 SFM_SIDEWARD=0.4 \
./bin/basic_sfm datasets/test_data1.txt cloud1_combo.ply
```

5. The automatic sweep script

./run_experiments.sh runs four sweeps on both datasets and prints a tabulated summary of point counts and registered cameras per configuration. It writes one .ply per cell into *results/* and one log per cell into *results/_logs/*.

```
chmod +x run_experiments.sh          # one time only
./run_experiments.sh
```

The three sweeps are:

EXPERIMENT 1. Loss kernel comparison (Cauchy vs Huber vs NULL) at the default scale 2.0. Directly satisfies the spec's request to *compare the standard NULL loss with robust alternatives*.

EXPERIMENT 2. Scale sweep for both Cauchy and Huber at scales 1.0, 1.5, 2.0, 3.0, 4.0.

EXPERIMENT 3. Sideward threshold sweep for task 3/7 at 0.3, 0.4, 0.5, 0.6, 0.7.

Total runtime: 5 to 15 minutes on a recent laptop, depending on the number of seeds the BA loop has to try and on how often Cauchy/Huber settle quickly.

6. The fundamental limit (read this)

All the knobs above operate on the data the matcher already produced. Tasks 1/7 and 2/7 (which produce the matches) are NOT part of this minimal submission, so the input *test_data1.txt* and *test_data2.txt* are fixed, and the set of correspondences they encode is fixed too. No knob inside tasks 3-7 can extract more information than is present in the matches.

Concretely, the upper bound on density is the number of unique 3D points implied by the matches: 9 759 for DS1 and 6 744 for DS2 (the second number on the *Header* line basic_sfm prints). Default Cauchy@2.0 reaches roughly 35 to 60 percent of those upper bounds; Huber at large scale gets

closer to 80 percent on DS1 but at the cost of admitting noisier 3D points.

If the visual quality of the cloud is unsatisfactory at every configuration you try, the limitation is the matcher, not the incremental SfM. A denser and more shape-faithful reconstruction would require either implementing tasks 1/7 + 2/7 with a stronger feature detector (SIFT, DISK, SuperPoint), or a multi-view stereo post-processing step. Both are outside this minimal submission's scope.

7. Recommended workflow

Step 1. Build (section 1).

Step 2. Run the default command and inspect *cloud1.ply* and *cloud2.ply* in MeshLab. If you are happy with both, you can stop here; this is also exactly what the minimal submission produces.

Step 3. If a cloud looks too sparse, run the sweep script and pick the configuration whose .ply looks best to you. Cross-check with the printed table: the column *points* shows the density of each cell.

Step 4. Once you have decided which configuration is best, you have two ways to commit to it:

(a) Submit this experimentation folder as-is. The defaults are compliant with the spec; the env vars are an opt-in feature, not a spec violation.

(b) Hardcode the chosen values inside *basic_sfm.cpp* (replace the *std::getenv* reads with constants) and submit as a single canonical configuration.

8. Spec-compliance notes

All three knobs default to values either explicitly recommended by the lab spec, or that satisfy its qualitative requirements. The default behaviour of the binary, when invoked without environment variables, matches the minimal submission byte-for-byte in its decision logic.

Per-task spec mapping at default settings:

Task 3/7. Spec says "check whether the recovered transformation is primarily a sideward motion". With `SFM_SIDEWARD=0.5` (default), we reject pairs where both $|tx|$ and $|ty|$ are below 0.5.

Task 6/7. Spec says "experiment with different robust loss kernels: `ceres::HuberLoss(a)` or `ceres::CauchyLoss(a)` where *a* is a scale parameter (e.g., 1.0 or $2 * \text{max_reproj_err}$)". With `SFM_LOSS=cauchy` and `SFM_SCALE=2.0` (defaults), we use exactly `ceres::CauchyLoss(2.0 * max_reproj_err)`, one of the two spec-listed examples.

Task 7/7. Spec says "check at each registration iteration if the reconstruction has diverged". The divergence detector uses two checks: catastrophic point loss ($n_valid_pts < 10$) and any registered camera with translation outside `max_dist`. No env-var knob; the threshold uses only quantities from the surrounding template code.

OPTIONAL 1 and OPTIONAL 2 are NOT touched. Their template blocks inside *basic_sfm.cpp* are byte-identical to the lab template, verified by automated diff.

9. File layout

CMakeLists.txt: unchanged from template

README.txt: unchanged from template

HOW_TO_USE.pdf: this document

run_experiments.sh: sweep runner script

src/basic_sfm.cpp: tasks 3 to 7 implemented; OPTIONAL 1+2 untouched; knobs at `SFM_LOSS`, `SFM_SCALE`, `SFM_SIDEWARD`

src/features_matcher.cpp: unchanged template (tasks 1+2 NOT implemented)

src/*.h, io_utils.cpp: unchanged from template

src/matcher_app.cpp, sfm_app.cpp: unchanged (do-not-modify per spec)

datasets/3dp_cam.yml: calibration

datasets/test_data{1,2}.txt: shipped pre-computed matcher outputs (tasks 1+2 not implemented)

results/: output directory for produced .ply files